CryptoChain Analyzer Dashboard

Hash Functions, Proof of Work, Difficulty, AI Anomaly Detection and Security Analysis

Final Extended Technical Report

Student	Claudia Corona Blanco
GitHub username	claudiacorona065
Repository	EuropeanValley/blockchain-dashboard-claudiacorona065
Course	Cryptography - Universidad Alfonso X el Sabio
Academic year	2025-2026
Dashboard framework	Streamlit
Core modules	M1 Proof of Work Monitor, M2 Block Header Analyzer, M3 Difficulty History, M4 AI Component
Optional extension	M6 Security Score

Executive positioning

This report explains not only what was implemented, but why each design decision was made, how the dashboard connects live Bitcoin data to cryptographic theory, how the AI component was evaluated, and how the optional security module extends the work beyond the minimum requirements.

May 2026

Table of Contents

1. Executive Summary
2. Project Requirements and How They Were Addressed
3. Dashboard Architecture and Data Pipeline
4. M1 - Proof of Work Monitor
5. M2 - Block Header Analyzer
6. M3 - Difficulty History
7. M4 - AI Component: Anomaly Detection
8. M6 - Optional Security Score
9. Real-Time Behaviour, Caching and Error Handling
10. Visualisation Strategy and User Experience
11. Main Findings, Limitations and Future Work
12. Evaluation Against the Rubric

1. Executive Summary

The CryptoChain Analyzer Dashboard is an interactive Python application that connects to public Bitcoin APIs and transforms live blockchain data into cryptographic, statistical and security-oriented evidence. The project was developed as a Streamlit dashboard with separate tabs for each module, following the repository structure required by the assignment: app.py as the entry point, api/ as the data access layer, modules/ as the modular analysis layer, requirements.txt for dependencies, and a final report folder for documentation.

The final dashboard implements all required modules M1-M4 and one optional extension, M6. The core modules cover Proof of Work monitoring, block header reconstruction, local Proof of Work verification, difficulty retargeting analysis and an AI-based anomaly detector. The optional M6 Security Score goes beyond the minimum specification by estimating the energy-only lower-bound cost of a 51% attack and visualising how confirmation depth reduces the probability of a double-spend catch-up scenario.

The main design principle was to avoid treating blockchain values as isolated numbers. Every displayed metric is connected to a cryptographic concept: the block hash is connected to SHA-256 and leading zero bits; the bits field is converted into a target threshold; the nonce is interpreted as a search variable; difficulty is connected to the expected amount of hash work; block intervals are connected to the stochastic nature of mining; and confirmation depth is connected to probabilistic security.

Final status

The required core is complete: M1, M2, M3 and M4 are implemented in the dashboard. The dashboard also includes automatic refresh and the optional M6 Security Score. The repository history shows progressive development through meaningful commits rather than a last-minute upload.

2. Project Requirements and How They Were Addressed

The assignment required a real-time dashboard that uses a public blockchain API, displays live cryptographic metrics from Bitcoin and integrates an AI component that adds analytical value. The dashboard was implemented with Streamlit because it provides a direct way to create interactive Python data applications while keeping the project understandable and maintainable. The main output is not a command-line script: it is a browser-based dashboard with tabs, sliders, tables, charts and explanatory panels.

The project brief defined four required modules and optional extensions for higher-grade levels. The implementation covers each required element and adds the optional M6 module. The following matrix summarises the mapping from specification to implementation.

Requirement	What the brief asked for	How the dashboard addresses it
M1 Proof of Work Monitor	Show current difficulty, leading-zero threshold, last N block times, distribution and estimated hash rate.	Implemented in a dedicated Streamlit tab with live metrics, target conversion, leading zero visualisation, interval histogram and recent block table.
M2 Block Header Analyzer	Show the six block header fields, reconstruct the 80-byte header, compute double SHA-256 using hashlib and verify Proof of Work.	Implemented with local header serialization, byte-order handling, SHA256(SHA256(header)), hash comparison, target comparison and PASS/FAIL verification table.
M3 Difficulty History	Plot difficulty over adjustment periods, mark adjustment events and compute actual/target block-time ratio.	Implemented using 2016-block boundaries, difficulty derived from bits, ratio actual average block time / 600 seconds, line and bar charts, and period table.
M4 AI Component	Choose and evaluate one AI approach.	Implemented as an unsupervised anomaly detector for Bitcoin block inter-arrival times using an exponential distribution baseline, train/test split, anomaly rate and negative log-likelihood.
M6 Optional Security Score	Estimate cost of a 51% attack and visualise confirmation-depth risk.	Implemented as an optional module with live hash rate, configurable hardware/electricity assumptions, energy cost estimate and Nakamoto-style confirmation probability chart.
Real-time dashboard	The dashboard must update automatically.	Implemented with global auto-refresh using a configurable 60-second polling interval and

Requirement	What the brief asked for	How the dashboard addresses it
		cached API calls to reduce unnecessary requests.

3. Dashboard Architecture and Data Pipeline

The implementation follows a deliberately modular architecture. The file app.py is responsible for page configuration, global dashboard controls and tab navigation. The api/blockchain_client.py file centralises all API calls and shared cryptographic helper functions. Each module is implemented in a separate file under modules/, which keeps the code readable and allows each tab to be developed, tested and explained independently.

This separation is important because the project is not only a dashboard but also an educational cryptography artefact. The API layer fetches raw data. The module layer transforms that raw data into cryptographic evidence. The Streamlit layer presents it in a way that can be understood without needing to inspect the source code.

File	Responsibility	Why it exists
app.py	Dashboard entry point	Defines page configuration, sidebar controls, automatic refresh, and the tab layout for M1, M2, M3, M4 and M6.
api/blockchain_client.py	Data and utility layer	Connects to Blockchain.com and Blockstream APIs; implements bits-to-target conversion, leading zero counting, recent block retrieval, hash-rate helpers and difficulty helpers.
modules/m1_pow_monitor.py	Proof of Work monitoring	Fetches current/recent block data and presents mining metrics, target, leading zero bits, interval distribution and hash rate estimate.
modules/m2_block_header.py	Block header verification	Reconstructs the 80-byte Bitcoin header and verifies the block hash locally with hashlib.
modules/m3_difficulty_history.py	Difficulty retargeting analysis	Uses block heights at 2016-block boundaries to analyse actual block times and difficulty adjustments.
modules/m4_ai_component.py	AI anomaly detector	Fits an exponential baseline to inter-arrival times and flags statistically unusual intervals.
modules/m6_security_score.py	Optional security extension	Estimates energy-only 51% attack cost and plots double-spend catch-up probabilities.

Two public data sources are used. Blockchain.com is used for aggregated data such as current difficulty and raw block details. Blockstream/Esplora is used for efficient block lookup by height and lightweight block metadata. This combination avoids depending on one endpoint for everything and makes it possible to analyse both the latest block and historical adjustment boundaries.

Endpoint/source	Information used	Reason for use
Blockchain.com latestblock/rawblock	Latest block summary and full raw block details.	Needed for M1 and M2 because the dashboard needs the current block hash, nonce, bits, transaction count and full header-related fields.
Blockchain.com getdifficulty	Current network difficulty as a decimal number.	Used for live difficulty display and hash-rate estimation.
Blockstream /block-height/:height	Convert a block height into a block hash.	Used in M3 and M6 when analysing exact block heights at difficulty boundaries or recent sequences.
Blockstream /block/:hash	Lightweight block metadata.	Used for timestamp, height, tx_count, bits, nonce and difficulty without downloading every transaction.

4. M1 - Proof of Work Monitor

M1 is the dashboard entry point into live Bitcoin mining. Its goal is to make the current state of Proof of Work visible: how difficult it is to mine, how strict the current target is, how many leading zero bits appear in the latest block hash, how variable recent block times are, and what the implied network hash rate is.

4.1 What was implemented

- A slider to select how many recent blocks should be analysed.
- A live mining overview with latest block height, current difficulty, average block time and estimated network hash rate.
- A latest block Proof of Work section showing nonce, transaction count, bits and the block hash.
- A target threshold derived from the bits field and displayed as a 256-bit hexadecimal threshold.
- A leading-zero-bits progress visualisation that compares the observed hash with the target requirement.
- A histogram of time between recent blocks with a reference line at the Bitcoin target of 600 seconds.
- A recent block table showing height, timestamp, hash, nonce, bits and transaction count.

4.2 Why each metric matters

The current difficulty is not simply a dashboard number; it is a protocol-level measure of how much work is expected to find a valid block. A high difficulty means that fewer hashes fall below the current target, so miners must try more nonce/header combinations on average. The block hash is displayed because Proof of Work is visually recognisable through its leading zeros, but the actual rule is numeric: the 256-bit hash interpreted as an integer must be below the target.

The nonce is included because it is the most familiar variable miners change while searching for a valid block. In practice, miners also vary other header-related data, such as extra nonce information through the coinbase transaction and Merkle root, but the nonce remains a clear way to explain the repeated search process. The transaction count is included to connect the cryptographic block header to the economic content of the block.

Recent block intervals are shown because Bitcoin does not produce a block exactly every ten minutes. Mining is a probabilistic process. A histogram is appropriate because the objective is to show the distribution of waiting times, not a single deterministic schedule. The expected baseline is an exponential-like waiting-time distribution under stable hash rate assumptions, which later connects directly to the AI anomaly detector in M4.

```
hash_rate ≈ difficulty × 2^32 / average_block_time
```

Used in M1 and M6 to estimate the network hash rate from live difficulty and observed block intervals.

4.3 Conclusions from M1

M1 demonstrates that Proof of Work can be observed directly from public blockchain data. The leading zeros in the hash provide an intuitive visual clue, but the dashboard goes further by computing the target from bits and comparing the result with the 256-bit hash space. The block interval histogram also shows that a single block can be much faster or slower than ten minutes without implying failure; the target is long-term average behaviour, not per-block determinism.

5. M2 - Block Header Analyzer

M2 is the strongest cryptographic verification module. It does not trust the API hash blindly. Instead, it reconstructs the Bitcoin block header locally, applies double SHA-256 with Python hashlib and verifies that the result matches the explorer hash and satisfies the Proof of Work target.

5.1 Header fields and 80-byte structure

A Bitcoin block header is exactly 80 bytes. The dashboard presents the six fields required by the assignment: version, previous block hash, Merkle root, timestamp, bits and nonce. The size breakdown is shown implicitly through the table and explicitly through the serialized header length:

```
4 + 32 + 32 + 4 + 4 + 4 = 80 bytes
```

Version + previous hash + Merkle root + timestamp + bits + nonce

Header field	Size	Meaning
Version	4 bytes	Signals block version/rules and is serialized little-endian.
Previous block hash	32 bytes	Links the block to its parent and creates the chain structure. Serialized in internal byte order.

Header field	Size	Meaning
Merkle root	32 bytes	Commits to all transactions in the block. Serialized in internal byte order.
Timestamp	4 bytes	Miner-provided Unix timestamp. Used carefully because timestamps are not perfect wall-clock data.
Bits	4 bytes	Compact representation of the Proof of Work target.
Nonce	4 bytes	Search variable changed by miners while attempting to find a valid hash.

5.2 Why byte order matters

The most important implementation detail in M2 is byte order. Bitcoin block explorers display hashes in a human-readable hexadecimal form, but the serialized block header uses internal byte order for some fields. The implementation reverses the previous block hash and the Merkle root byte-by-byte before hashing. It also serializes integer fields as 4-byte little-endian values. Without these conversions, the locally calculated hash would not match the API hash.

5.3 Local Proof of Work verification

The dashboard builds the serialized header, computes SHA256(SHA256(header)), reverses the digest for standard explorer display and compares it against the API hash. It then converts the calculated hash into an integer and verifies that it is lower than or equal to the target derived from bits. This is the key step that connects the dashboard to real cryptographic validation rather than simply visualising API output.

```
calculated_hash = reverse(SHA256(SHA256(serialized_header)))
```

The reverse is applied to display the hash in the conventional Bitcoin explorer format.

```
valid Proof of Work ⇔ int(calculated_hash, 16) ≤ target
```

This is the numeric condition behind the visible leading zeros.

5.4 Conclusions from M2

M2 proves that the dashboard understands the cryptographic structure of a Bitcoin block. The block hash is not accepted as a label; it is reproduced from the underlying header. The PASS/FAIL verification table makes the verification explicit: header length is 80 bytes, local double SHA-256 matches the API hash and the calculated hash is below the target. This directly satisfies the highest-level cryptographic correctness requirement of the rubric.

6. M3 - Difficulty History

M3 analyses the mechanism that keeps Bitcoin block production near a ten-minute long-term average. Instead of plotting arbitrary historical difficulty points, the dashboard focuses on completed difficulty adjustment periods of 2016 blocks. This is important because the protocol retargets difficulty at these boundaries, so each marker on the difficulty chart has protocol meaning.

6.1 What was implemented

- A slider to choose the number of completed adjustment periods to analyse.
- Calculation of the latest completed 2016-block boundary from the current chain height.
- Block lookup at boundary heights using Blockstream metadata.
- Difficulty calculation from the bits field at each period start.
- Actual period duration using timestamps of boundary blocks.
- Average block time per period and ratio against the 600-second target.
- Line chart of difficulty evolution at adjustment boundaries.
- Bar chart of actual/target block-time ratio per period.
- Detailed table with dates, bits, difficulty, percentage change, actual days, expected days, average block time and interpretation.

6.2 Retargeting logic and why the ratio matters

Each difficulty period contains 2016 blocks. If blocks were found exactly every 600 seconds, the period would last 2016 × 600 seconds, which is approximately 14 days. The dashboard therefore computes actual period duration and compares it with the expected duration. The ratio actual_average_block_time / 600 is easier to interpret than raw seconds because it directly tells whether the network was faster or slower than target.

```
actual_target_ratio = average_block_time_seconds / 600
```

Ratio < 1 means blocks were faster than target; ratio > 1 means blocks were slower than target.

When the ratio is below 1, the network found blocks faster than expected. That usually creates upward difficulty pressure at the next adjustment because the protocol needs to make future blocks harder. When the ratio is above 1, blocks were slower than expected, producing downward difficulty pressure. This makes the chart educational rather than purely descriptive.

6.3 Conclusions from M3

M3 shows that difficulty is not a static security parameter. It is a feedback mechanism. The dashboard makes this visible by aligning the time series with actual protocol boundaries and by presenting both difficulty and timing ratios. The main conclusion is that Bitcoin self-corrects mining speed over long windows: individual blocks may be random, but difficulty adjustment stabilises the expected production rate over completed periods.

7. M4 - AI Component: Anomaly Detection

For the AI component, the selected approach was an unsupervised anomaly detector for abnormal Bitcoin block inter-arrival times. This choice was made because it is directly connected to the cryptographic mining process already analysed in M1 and M3. Bitcoin block discovery is a repeated random search for a hash below target. Under stable network hash-rate assumptions, the waiting time between block discoveries can be modelled with an exponential distribution. Therefore, an exponential baseline is a defensible statistical model for identifying unusually short or unusually long intervals.

7.1 Model choice and justification

The anomaly detector is preferable to a more complex black-box model for this project because it is transparent, explainable and mathematically connected to Proof of Work. The model does not require labelled examples of attacks or pool behaviour. Instead, it learns a baseline from recent real block intervals and identifies intervals that are unlikely under that baseline. This makes it suitable for an educational cryptography dashboard: the user can understand the assumptions, the thresholds and the limitations.

Chosen AI approach

Anomaly detector for abnormal block inter-arrival times using an exponential distribution baseline. This was selected because it is explainable, uses real Bitcoin data, and connects naturally to the probabilistic behaviour of Proof of Work mining.

7.2 Data used by the model

The model uses recent Bitcoin block metadata obtained from the Blockstream API. For each adjacent pair of blocks, the dashboard computes the time difference between timestamps. Each interval is associated with the destination block height, block hash, timestamp, transaction count, difficulty, nonce and bits. These variables are displayed or kept for interpretability, but the core model is trained on the interval seconds.

A specific design decision was made to handle non-positive timestamp intervals separately. Bitcoin block timestamps are miner-provided and are not guaranteed to be strictly increasing between adjacent blocks. If a non-positive interval appears, the dashboard flags it as a timestamp irregularity rather than treating it as a normal exponential-model anomaly. This prevents timestamp artefacts from contaminating the evaluation of the statistical model.

7.3 Training and evaluation design

The model uses a simple train/test split. The oldest 70% of selected recent valid intervals are used to fit the exponential rate parameter, and the newest 30% are used for evaluation. This prevents the evaluation metrics from being computed on exactly the same data used to fit the baseline.

```
λ = 1 / mean(training_intervals)
```

Maximum likelihood estimate of the exponential rate parameter.

```
anomaly_score = -log10(tail_probability)
```

Large scores correspond to intervals that are less likely under the fitted exponential baseline.

The sensitivity parameter alpha controls how strict the detector is. With alpha = 0.05, roughly 5% of observations are expected to fall outside the normal region under the model. The dashboard computes lower and upper thresholds for very fast and very slow blocks using a two-tail test. This means the model can detect both extremes: blocks that arrive surprisingly soon and blocks that take surprisingly long.

7.4 Metrics displayed

Metric	Definition	Why it matters
Training mean interval	Average interval learned from the training data.	Shows the fitted baseline in human terms.
Test anomaly rate	Fraction of test intervals flagged as anomalous.	Evaluates whether the detector is producing a plausible number of alerts.
Average test negative log-likelihood	Average negative log probability of test intervals under the model.	Lower values mean the exponential baseline fits the test intervals better.
Tail probability	Probability mass in the relevant tail for an interval.	Explains why a specific interval was or was not flagged.
Anomaly score	-log10(tail probability).	Converts rare-event probability into a readable score.

7.5 Visual outputs and conclusions

M4 contains two main visualisations. The scatter plot places block height on the x-axis and seconds since previous block on the y-axis. Horizontal reference lines show the 600-second Bitcoin target and the fast/slow anomaly thresholds. This makes it easy to identify which blocks are ordinary variability and which ones cross the statistical threshold. The histogram shows the empirical distribution of recent intervals and overlays the target and anomaly thresholds, connecting observed data to the model.

The main conclusion is that unusual block intervals are expected in a stochastic Proof of Work system; the AI component does not claim to prove attacks. Instead, it provides a statistically grounded alerting layer. The limitation section in the dashboard explicitly states that detected anomalies are evidence for investigation, not definitive security incidents. This is important for academic correctness and professional interpretation.

8. M6 - Optional Security Score

M6 was added because the optional modules are designed to reach the highest grade levels. It extends the dashboard beyond core course content by connecting Proof of Work metrics to economic security. The module estimates an energy-only lower-bound cost for a 51% attack and visualises how confirmation depth affects double-spend catch-up probability.

8.1 What was implemented

- A configurable number of recent blocks to estimate average block time and network hash rate.
- Inputs for mining efficiency in J/TH and electricity price in USD/kWh.
- A configurable attacker share after joining the network, defaulting to 51%.
- Estimated current network hash rate using live difficulty and observed block times.
- Estimated attacker hash rate needed to obtain the selected share of total hash power.
- Estimated attacker power requirement and electricity cost per hour.
- A bar chart comparing the lower-bound energy cost of the current network and the attacker scenario.
- A line chart showing double-spend catch-up probability across confirmation depths for several attacker shares.
- A security interpretation table explaining 51% attack condition, cost estimate, confirmations and limitations.

8.2 Hash-rate and attack-cost formulas

```
H ≈ difficulty × 2^32 / average_block_time
```

Estimated honest network hash rate H in hashes per second.

```
A = q / (1 - q) × H
```

Attacker hash rate A required for attacker share q after joining the honest network.

```
power(W) = (hash_rate / 10^12) × efficiency(J/TH)
```

Since J/s = W, TH/s multiplied by J/TH gives watts.

```
cost_per_hour = power(W) / 1000 × electricity_price(USD/kWh)
```

Energy-only lower-bound cost per hour.

The module intentionally labels the cost estimate as a lower bound. It excludes hardware acquisition, ASIC supply constraints, cooling, facilities, pool coordination, operational risk, market reaction and detection. This prevents the number from being misinterpreted as a realistic full attack budget. The purpose is educational: to show the scale implied by live hash-rate data and to connect hash power with economic security.

8.3 Confirmation depth and Nakamoto-style probability

The confirmation-depth chart implements the intuition from Nakamoto's double-spend analysis. If an attacker has less hash power than the honest network, the probability of catching up decreases as the honest chain gains more confirmations. The dashboard plots several attacker shares, such as 10%, 20%, 30%, 40% and 49%, to show how risk changes with attacker strength.

This visualisation is valuable because it explains why users wait for confirmations. Confirmations are not arbitrary delays; they increase the amount of Proof of Work an attacker must overcome. The chart also shows why the case close to 49% remains dangerous for longer: the attacker is almost as powerful as the honest network, so catch-up probability declines much more slowly.

8.4 Conclusions from M6

M6 connects cryptographic security to economic and probabilistic security. The dashboard shows that Bitcoin security is not only about hash functions being hard to invert; it is also about the amount of energy and hardware implied by the current global hash rate. By combining cost estimation with confirmation-depth probability, M6 provides a broader security interpretation than the required modules alone.

9. Real-Time Behaviour, Caching and Error Handling

The final dashboard includes a global automatic refresh control in the sidebar. By default, the refresh interval is set to 60 seconds, which matches the high-level functionality expectation for automatic polling. The user can enable or disable automatic refresh and adjust the interval. The dashboard also displays the last interface refresh time and the auto-refresh count, making the behaviour visible rather than hidden.

Because public blockchain APIs should not be spammed, the dashboard uses Streamlit caching with time-to-live values. Caching reduces repeated requests during reruns while still allowing data to update. Manual refresh buttons are kept in modules so that the user can clear cached data intentionally when needed. This is a balance between real-time behaviour and responsible API use.

Each module uses exception handling around API calls. If an API request fails, the dashboard shows an error message through the interface instead of crashing the whole application. This matters because public APIs can be slow, rate-limited or temporarily unavailable.

Feature	Implementation	Reason
Global auto-refresh	streamlit-autorefresh with a configurable 60-second default.	Improves real-time behaviour and addresses the functionality criterion.

Feature	Implementation	Reason
Cached data loading	st.cache_data with TTL values around expensive API calls.	Avoids unnecessary repeated API requests during automatic reruns.
Manual refresh buttons	Module-level refresh buttons clear cached data.	Gives user control while preserving automatic behaviour.
Error handling	try/except blocks with st.error messages.	Prevents a single API failure from crashing the dashboard.

10. Visualisation Strategy and User Experience

The dashboard uses a tabbed structure so that each concept has its own space. This avoids mixing Proof of Work verification, difficulty history, AI modelling and security analysis in a single crowded page. The visual design follows a consistent pattern: overview metrics first, then detailed charts, then tables and finally explanatory expanders or interpretation text.

Module	Visual elements	Why this chart type fits
M1	Metrics, progress indicator, histogram, recent block table.	A histogram is appropriate for block intervals because the question is distributional.
M2	Header field table, code-style hexadecimal blocks, verification table, progress indicator.	Structured tables and code blocks are appropriate for serialized cryptographic data.
M3	Line chart, bar chart and period table.	A time series is appropriate for difficulty evolution; a bar chart is appropriate for period ratios.
M4	Scatter plot, histogram, anomaly table and metric table.	A scatter plot highlights anomalous intervals by block height; the histogram compares empirical intervals with model thresholds.
M6	Metric cards, cost bar chart, confirmation probability line chart and interpretation table.	Cost comparison and probability decay are easier to understand visually than as raw formulas.

The interface is designed so that a viewer can understand the purpose of each tab without reading the source code. Text boxes explain the meaning of thresholds, ratios and model outputs. Warning boxes are used where a result could be misinterpreted, especially in M4 timestamp irregularities and M6 attack-cost limitations.

11. Main Findings, Limitations and Future Work

11.1 Main findings

- Bitcoin Proof of Work can be verified locally from public block data by reconstructing the 80-byte header and applying double SHA-256.
- The bits field is central: it compactly encodes the target threshold, which determines whether a block hash is valid.
- Leading zeros are an intuitive visual symptom of Proof of Work, but the real rule is a numeric comparison between hash and target.
- Block intervals are variable because mining is probabilistic; the 600-second target is a long-term average, not a per-block guarantee.
- Difficulty retargeting stabilises the long-term block interval by adjusting every 2016 blocks.
- An exponential baseline is a transparent and suitable model for detecting unusual inter-arrival times, provided timestamp limitations are acknowledged.
- The optional 51% attack module shows that hash-rate security has an economic dimension and that confirmation depth reduces catch-up probability for sub-50% attackers.

11.2 Limitations

- All live values depend on public API availability and response formats.
- Hash-rate estimates are approximate because they infer global computation from difficulty and observed block times.
- Miner-provided timestamps are not perfect wall-clock measurements, so M4 separates timestamp irregularities from statistical anomalies.
- The M6 cost estimate is energy-only and should not be treated as a complete real-world attack budget.
- The AI model is intentionally simple and explainable; a more advanced model could incorporate longer history, difficulty regimes, mempool congestion or mining pool data.

11.3 Future work

- Add M5 Merkle Proof Verifier to verify an individual transaction path step by step.
- Add cached fallback data for demonstrations if a public API is temporarily unavailable.
- Export dashboard snapshots automatically for reproducible reporting.
- Compare the exponential anomaly detector with a second AI method, such as an isolation forest or a difficulty predictor.
- Add fee-market data from mempool.space to connect mining incentives with transaction demand.

12. Evaluation Against the Rubric

The following table summarises how the final project addresses the grading rubric. This is included to make the contribution explicit and to show that the implementation decisions were aligned with the assignment criteria.

Rubric criterion	Where addressed	Evidence	Assessment
C1 Cryptographic Correctness	M1, M2, M3	Manual double SHA-256 verification, 80-byte header reconstruction, bits-to-target conversion, leading zero bits, difficulty and retargeting analysis.	Very strong
C2 AI Component	M4	Real blockchain data, exponential anomaly detector, train/test split, anomaly rate, negative log-likelihood, limitations.	Strong
C3 Functionality and Real-Time Updates	app.py, all modules	Interactive Streamlit dashboard, API integration, auto-refresh at 60 seconds, caching and error handling.	Strong
C4 Visualisation	M1-M6	Appropriate chart types, labelled axes, metric cards, tables, explanations and interpretation boxes.	Strong
C5 Research and Report	M6 and this report	Nakamoto double-spend analysis, public API documentation, optional security module beyond course minimum, progressive commits.	Strong

13. References

[1] Satoshi Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf

[2] Blockstream / Esplora API documentation. https://github.com/blockstream/esplora/blob/master/API.md

[3] Blockchain.com Charts - Network Difficulty. https://www.blockchain.com/charts/difficulty

[4] Blockchain.com Developer APIs - q/getdifficulty and related endpoints. https://www.blockchain.com/api/q

[5] Streamlit documentation - st.set_page_config, st.tabs and st.cache_data. https://docs.streamlit.io/develop/api-reference

[6] Course project brief: CryptoChain Analyzer Dashboard, Prof. Jorge Calvo, Universidad Alfonso X el Sabio, Academic Year 2025-26.

Appendix A. Dashboard Inventory

This appendix lists the visible content created in each dashboard tab. It is included so that the evaluator can understand the full scope of the work even without clicking through every chart during a live demonstration.

Tab	Dashboard section	What it demonstrates
M1	Live mining overview	Latest block height, current difficulty, average block time, estimated hash rate.
M1	Latest block PoW data	Nonce, transaction count, bits and latest block hash.
M1	Target and leading zeros	Target threshold derived from bits and observed leading zero bits in the latest hash.
M1	Block interval distribution	Histogram of recent block times with 600-second target reference.

Tab	Dashboard section	What it demonstrates
M1	Recent block table	Height, time, hash, nonce, bits and transaction count for recent blocks.
M2	Block overview	Height, version, nonce and bits for selected latest/custom block.
M2	Hash verification	API hash compared with locally calculated double SHA-256 hash.
M2	Header structure	Six fields, sizes, human-readable values and serialized header values.
M2	Proof of Work verification	PASS/FAIL checks for header length, hash match and target comparison.
M3	Difficulty overview	Current height, periods analysed, latest average block time and ratio.
M3	Difficulty chart	Line chart at recent adjustment boundaries.
M3	Ratio chart	Actual average block time divided by 600-second target per period.
M3	Period table	Dates, bits, difficulty, percentage change, actual days, expected days and interpretation.
M4	Model configuration	Recent block count and anomaly sensitivity alpha.
M4	AI overview	Blocks analysed, training mean interval, test anomaly rate and average test NLL.
M4	Anomaly chart	Scatter plot of intervals with target and anomaly threshold lines.
M4	Distribution chart	Histogram of block intervals with model thresholds.
M4	Anomaly table	Detected statistical anomalies and timestamp irregularities.
M4	Evaluation table	Training intervals, test intervals, expected anomaly rate, observed anomaly rate and NLL.
M6	Assumptions	Recent block count, mining efficiency, electricity price and attacker share.
M6	Security overview	Network hash rate, attacker hash rate, power estimate and energy cost per hour.
M6	Energy cost comparison	Bar chart comparing current network and attacker scenario.
M6	Confirmation probability	Line chart of double-spend catch-up probability by confirmation depth.
M6	Interpretation table	51% attack condition, cost estimate, confirmations and limitations.